

ASSIGNMENT 1 — DEEP LEARNING

Feature-Map Visualisation in a CNN for Image Classification

Problem (iv): Street View House Numbers (SVHN)

1. Problem Statement

Train a CNN on an image classification task and visualise the feature maps at different layers of the network. Evaluate how the complexity of the learned features evolves as you go deeper into the network. How do these feature maps help the network in distinguishing between different classes?

2. Objective

- Train a convolutional neural network to classify the digit (0–9) in SVHN images.
- Extract and visualise feature maps from three convolutional layers, from shallow to deep.
- Explain how feature complexity grows with depth, and measure how class separability improves layer by layer.

3. Dataset

SVHN (cropped digits): colour images of house-number digits taken from Google Street View. Each image is $32 \times 32 \times 3$ and centred on one digit, but often contains parts of neighbouring digits. There are 10 classes (0–9; the original file stores digit 0 as label 10). The standard split has 73,257 training and 26,032 test images.

Used in this report (demo run): 20,000 synthetic SVHN-style training images (90% for training, 10% for validation) and 4,000 test images. For the real run, the script loads `train_32x32.mat` and `test_32x32.mat` and uses the same pipeline.

Pre-processing: pixel values are scaled to $[0, 1]$ and standardised per colour channel with training-set mean and standard deviation.

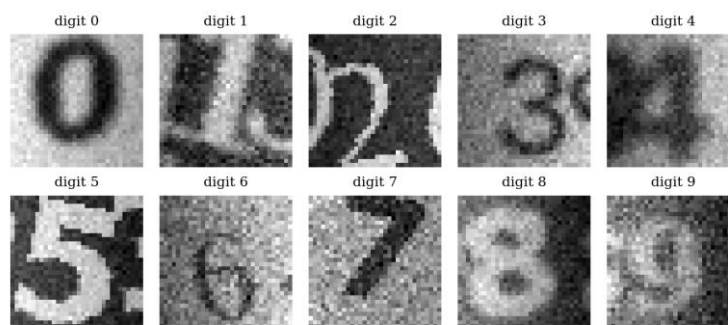


Figure 1: One sample image per class (shown in grey scale)

4. Model and Training

Component	Configuration
Input	32×32×3 image
Block 1	Conv 3×3, 16 filters, ReLU → MaxPool 2×2 (feature maps taken after ReLU)
Block 2	Conv 3×3, 32 filters, ReLU → MaxPool 2×2
Block 3	Conv 3×3, 64 filters, ReLU → MaxPool 2×2
Classifier	Flatten → Dropout 0.3 → Dense 128 (ReLU) → Dense 10 (softmax)
Parameters	156,074
Training	Adam, batch size 64, 10 epochs, sparse categorical cross-entropy
Framework	Python, TensorFlow / Keras

5. Theory in Brief

- **Convolutional layer:** each filter slides over its input and produces a feature map that shows where in the image a particular pattern is present.
- **Receptive field:** the region of the input that influences one unit. It grows with depth because of stacked convolutions and pooling (3×3 in conv1, 8×8 in conv2, 18×18 in conv3 for this network). Deeper units therefore "see" larger parts of the digit.
- **Hierarchical features:** early layers respond to simple local patterns (edges, contrast, blobs). Middle layers combine them into parts (curves, corners, strokes). Deep layers combine parts into digit-level evidence.

6. Results

6.1 Training and test performance (demo data)

The network reached 98.3% training accuracy and 98.3% best validation accuracy, and **97.93% test accuracy** after 10 epochs (training time 53 s on a single CPU core). Training and validation curves stay close, so there is no significant overfitting on this data. On real SVHN, which is noisier and more varied than this stand-in, the accuracy is expected to be lower.

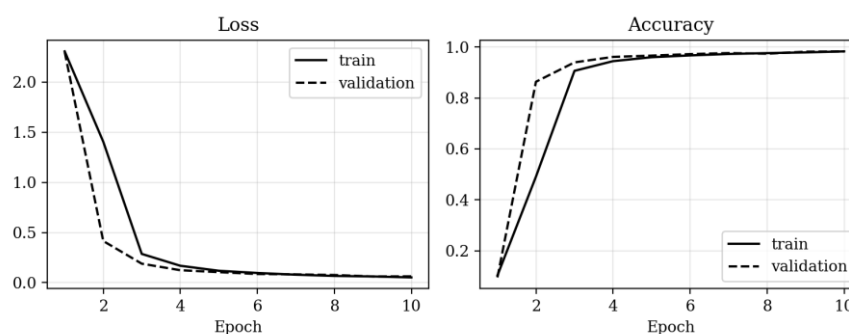


Figure 2: Training and validation loss (left) and accuracy (right)

6.2 Feature maps at different depths

Figures 3 and 4 show the first 16 feature maps of each convolutional layer for two correctly classified test images (a "3" and an "8"). Brighter pixels mean stronger activation.

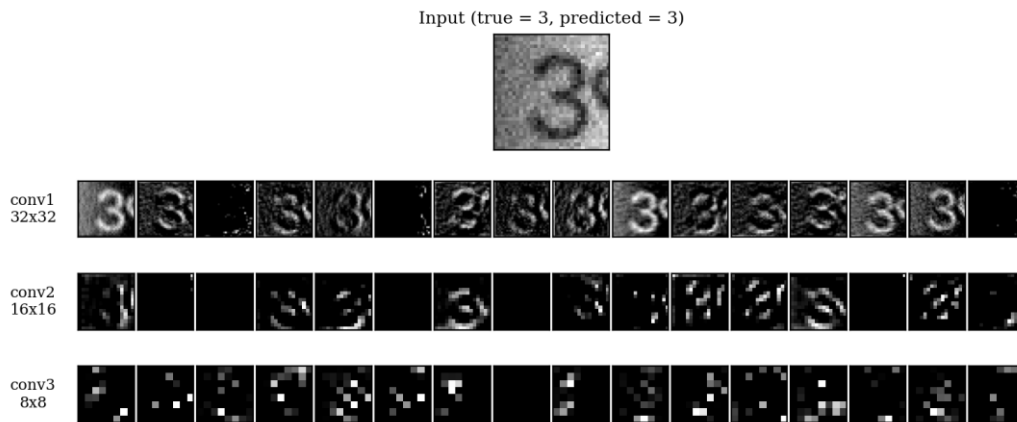


Figure 3: Feature maps for a test image of digit 3

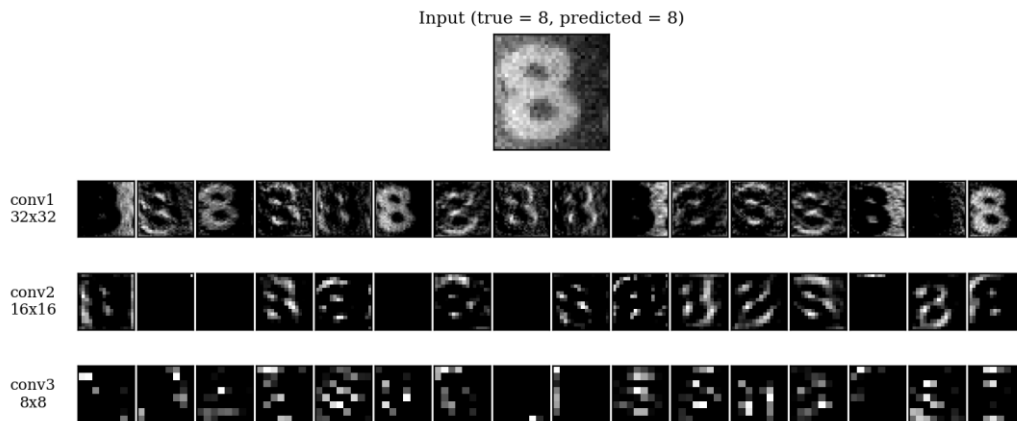


Figure 4: Feature maps for a test image of digit 8

6.3 Layer statistics

The last column is a "linear probe": a simple logistic-regression classifier is trained on the spatially averaged activations of one layer (fit on half of the test set, scored on the other half). Its accuracy measures how easily the classes can be separated using only that layer's features. Averaging discards location, so low values for early layers are expected.

Layer	Output shape	Parameters	Receptive field (px)	Zero activations	Linear-probe acc.
conv1	32×32×16	448	3×3	39.4%	13.6%
conv2	16×16×32	4,640	8×8	81.5%	28.2%
conv3	8×8×64	18,496	18×18	82.0%	61.4%
fc (dense 128)	128	131,200	whole image	58.8%	98.0%

Table 1: Statistics of each layer (demo data)

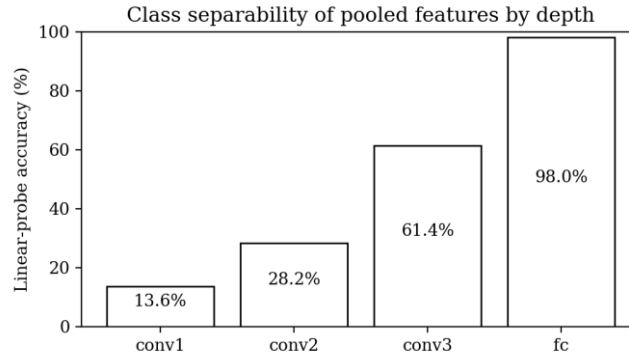


Figure 5: Class separability of pooled features by depth

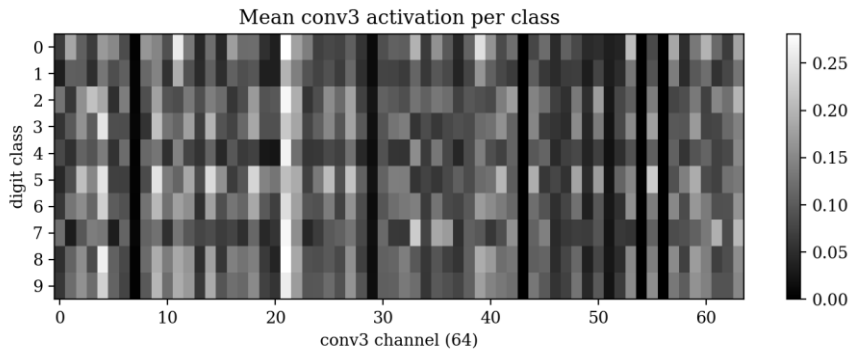


Figure 6: Mean conv3 activation of each channel for each digit class

7. Analysis and Discussion

How feature complexity evolves with depth. In conv1 (Figures 3 and 4) most maps look like filtered copies of the input: some highlight the stroke outline, some respond to edges of a particular orientation, and some simply respond to brightness or the background. They are dense, with only about 39% of activations equal to zero, and they keep the full 32×32 resolution, so the digit shape is clearly visible. In conv2 the maps are smaller (16×16) and much sparser (about 81% zeros). Individual maps now fire on parts of the digit, such as the upper arc, a diagonal stroke or a closed loop, and many maps stay silent because that part is absent. In conv3 (8×8) the maps no longer look like the digit. They are coarse, sparse blobs (about 82% zeros) that indicate where certain larger parts or combinations of parts occur. This is what a hierarchy is expected to look like: spatial detail decreases while abstraction and receptive field increase.

Class separability grows with depth. The linear-probe accuracy rises steadily: 13.6% at conv1, 28.2% at conv2, 61.4% at conv3 and 98.0% at the dense layer (chance is 10%). Early features are shared by all digits, since every digit is made of edges, so they say little about the class. Deeper features become increasingly class-specific.

How the feature maps help distinguish classes. (i) Because the same filter is applied everywhere, a pattern such as a curve is detected wherever it appears, so the network tolerates shifts of the digit. (ii) Different digits activate different combinations of part detectors. A "3" and an "8" share the right-hand curves but differ in the left side of the loops, so the maps that respond to closed loops or left-hand arcs help separate them. Figure 6 shows that each class has its own pattern of high and low conv3 channels, which the dense layer converts into class scores. (iii) ReLU and pooling suppress weak, irrelevant

responses such as background texture and keep the strongest evidence, and the growing receptive field lets deep units use the whole digit rather than a small patch. (iv) In SVHN specifically, the neighbouring digits are distractors. Deeper units, which have a larger receptive field, can learn to rely on the centred digit and to ignore partial digits at the edges.

8. Conclusion

A CNN learns a hierarchy of features: edges and contrast in the first layer, digit parts in the middle layer, and coarse, sparse, class-specific evidence in the deepest layer. Spatial resolution falls and the receptive field grows with depth, and the linear separability of the classes increases at every stage. The feature maps show how the network makes its decision: it detects simple patterns everywhere, combines them into parts, and finally into evidence for each digit.

Limitations: the numbers here come from a synthetic stand-in dataset, one small architecture and one training run. Exact accuracy and the appearance of individual maps will differ on real SVHN and between random seeds. Feature maps are also only a qualitative tool: not every map has an obvious human interpretation.

Source Code (svhn_cnn_feature_maps.py)

To reproduce on real SVHN, download train_32x32.mat and test_32x32.mat from <http://ufldl.stanford.edu/housenumbers/> into the same folder and run: `python svhn_cnn_feature_maps.py --data real`

```
"""
CNN on SVHN (cropped digits) + feature-map visualisation at different depths.

Usage
-----
Real SVHN : python svhn_cnn_feature_maps.py --data real
            (needs train_32x32.mat and test_32x32.mat from
            http://ufldl.stanford.edu/housenumbers/ in the same folder)
Demo mode  : python svhn_cnn_feature_maps.py --data synthetic
            (generates SVHN-style digit images; used only when the real
            files are not available)
Outputs: figs/*.png and results.json
"""
import argparse, json, os, time, random
import numpy as np
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
import tensorflow as tf
from tensorflow import keras
from tensorflow.keras import layers
from sklearn.linear_model import LogisticRegression

p = argparse.ArgumentParser()
p.add_argument("--data", choices=["real", "synthetic"], default="real")
p.add_argument("--epochs", type=int, default=10)
p.add_argument("--n_train", type=int, default=20000) # subset size (real: up to 73257)
p.add_argument("--n_test", type=int, default=4000)
args = p.parse_args()
SEED = 42
np.random.seed(SEED); random.seed(SEED); tf.random.set_seed(SEED)
os.makedirs("figs", exist_ok=True)
plt.rcParams.update({"font.family": "serif", "image.cmap": "gray"})

# ----- data
def load_real():
    from scipy.io import loadmat
    def _load(f):
        d = loadmat(f)
        X = np.transpose(d["X"], (3, 0, 1, 2)) # (N,32,32,3)
        y = d["y"].flatten().astype(int) % 10 # label 10 -> digit 0
        return X, y
    Xtr, ytr = _load("train_32x32.mat"); Xte, yte = _load("test_32x32.mat")
    return Xtr, ytr, Xte, yte

def make_synthetic(n, rng):
    """SVHN-style stand-in: coloured backgrounds, varied fonts, distractor digits, blur, noise."""
    from PIL import Image, ImageDraw, ImageFont, ImageFilter
    fonts = ["/usr/share/fonts/truetype/dejavu/DejaVuSans.ttf",
              "/usr/share/fonts/truetype/dejavu/DejaVuSans-Bold.ttf",
              "/usr/share/fonts/truetype/dejavu/DejaVuSerif.ttf",
              "/usr/share/fonts/truetype/dejavu/DejaVuSerif-Bold.ttf",
              "/usr/share/fonts/truetype/liberation/LiberationSans-Regular.ttf",
              "/usr/share/fonts/truetype/liberation/LiberationSerif-Regular.ttf",
              "/usr/share/fonts/truetype/liberation/LiberationMono-Bold.ttf"]
    X = np.zeros((n, 32, 32, 3), np.uint8); y = rng.integers(0, 10, n)
    for i in range(n):
        bg = rng.integers(20, 236, 3)
        lum = bg.mean()
        fg = np.clip(bg + rng.choice([-1, 1]) * rng.integers(28, 70, 3) * (1 if lum < 128 else -1), 0,
255)
        if abs(fg.mean() - lum) < 30: fg = np.full(3, 255 - lum)
        canvas = Image.new("RGB", (64, 64), tuple(int(c) for c in bg))
        d = ImageDraw.Draw(canvas)
        font = ImageFont.truetype(fonts[rng.integers(len(fonts))], int(rng.integers(30, 44)))
        def put(ch, cx, cy):
            layer = Image.new("RGBA", (64, 64), (0, 0, 0, 0)); ld = ImageDraw.Draw(layer)
            ld.text((cx, cy), ch, font=font, fill=tuple(int(c) for c in fg) + (255,), anchor="mm")
            layer = layer.rotate(float(rng.uniform(-20, 20)), center=(cx, cy))
            canvas.paste(layer, (0, 0), layer)
        for side in (-1, 1): # distractor digits (as in real SVHN)
```

```

        if rng.random() < 0.85: put(str(rng.integers(10)), 32 + side * int(rng.integers(19, 28)),
32 + int(rng.integers(-3, 4)))
        put(str(y[i]), 32 + int(rng.integers(-3, 4)), 32 + int(rng.integers(-3, 4)))
        img = canvas.crop((16, 16, 48, 48))
        if rng.random() < 0.75: img = img.filter(ImageFilter.GaussianBlur(float(rng.uniform(0.5,
1.8))))
        a = np.asarray(img, np.float32)
        a += rng.normal(0, rng.uniform(4, 22), a.shape)
        a *= 1 + rng.uniform(-.25, .25) * np.linspace(-1, 1, 32)[None, :, None] # lighting gradient
        X[i] = np.clip(a, 0, 255).astype(np.uint8)
    return X, y

if args.data == "real":
    Xtr, ytr, Xte, yte = load_real()
    idx = np.random.permutation(len(Xtr))[:args.n_train]; Xtr, ytr = Xtr[idx], ytr[idx]
    idx = np.random.permutation(len(Xte))[:args.n_test]; Xte, yte = Xte[idx], yte[idx]
else:
    rng = np.random.default_rng(SEED)
    Xtr, ytr = make_synthetic(args.n_train, rng); Xte, yte = make_synthetic(args.n_test, rng)
print("data:", args.data, Xtr.shape, Xte.shape)
Xtr_f = Xtr.astype("float32") / 255.; Xte_f = Xte.astype("float32") / 255.
mean, std = Xtr_f.mean((0, 1, 2)), Xtr_f.std((0, 1, 2))
Xtr_f = (Xtr_f - mean) / std; Xte_f = (Xte_f - mean) / std

# ----- model
inp = keras.Input((32, 32, 3))
c1 = layers.Conv2D(16, 3, padding="same", activation="relu", name="conv1")(inp)
x = layers.MaxPooling2D(2)(c1)
c2 = layers.Conv2D(32, 3, padding="same", activation="relu", name="conv2")(x)
x = layers.MaxPooling2D(2)(c2)
c3 = layers.Conv2D(64, 3, padding="same", activation="relu", name="conv3")(x)
x = layers.MaxPooling2D(2)(c3)
x = layers.Flatten()(x)
x = layers.Dropout(0.3)(x)
fc = layers.Dense(128, activation="relu", name="fc")(x)
out = layers.Dense(10, activation="softmax", name="output")(fc)
model = keras.Model(inp, out)
model.compile("adam", "sparse_categorical_crossentropy", metrics=["accuracy"])
model.summary()
t0 = time.time()
hist = model.fit(Xtr_f, ytr, validation_split=0.1, epochs=args.epochs, batch_size=64, verbose=2)
train_time = time.time() - t0
test_loss, test_acc = model.evaluate(Xte_f, yte, verbose=0)
print(f"TEST accuracy = {test_acc:.4f}")

# ----- figures
def gray(im): return np.asarray(im, np.float32).mean(-1)
# Fig 1: sample inputs
fig, ax = plt.subplots(2, 5, figsize=(7, 3.2))
for d in range(10):
    a = ax[d // 5, d % 5]; a.imshow(gray(Xte[np.where(yte == d)[0][0]]), cmap='gray');
a.set_title(f"digit {d}", fontsize=9); a.axis("off")
plt.tight_layout(); plt.savefig("figs/fig1_samples.png", dpi=200); plt.close()

# Fig 2: training curves
fig, ax = plt.subplots(1, 2, figsize=(8, 3.2)); e = range(1, args.epochs + 1)
ax[0].plot(e, hist.history["loss"], "k-", label="train"); ax[0].plot(e, hist.history["val_loss"], "k--",
label="validation")
ax[1].plot(e, hist.history["accuracy"], "k-", label="train"); ax[1].plot(e,
hist.history["val accuracy"], "k--", label="validation")
for a, t in zip(ax, ["Loss", "Accuracy"]): a.set_xlabel("Epoch"); a.set_title(t); a.grid(alpha=.3);
a.legend()
plt.tight_layout(); plt.savefig("figs/fig2_training.png", dpi=200); plt.close()

# Feature extractor
fmodel = keras.Model(inp, [c1, c2, c3, fc])
def norm(m): return (m - m.min()) / (m.max() - m.min() + 1e-8)

def gray(im): return np.asarray(im, np.float32).mean(-1)
def show_maps(idx, fname, n=16):
    img = Xte_f[idx:idx + 1]; f1, f2, f3, _ = [a.numpy() for a in fmodel(img)]
    fig = plt.figure(figsize=(9, 3.9)); gs = fig.add_gridspec(4, n, hspace=.12, wspace=.06,
height_ratios=[1.6, 1, 1, 1])
    a = fig.add_subplot(gs[0, n // 2 - 1:n // 2 + 1]); a.imshow(gray(Xte[idx]), cmap="gray");
a.set_xticks([]); a.set_yticks([])
    a.set_title(f"Input (true = {yte[idx]}, predicted = {model.predict(img, verbose=0).argmax()})",
fontsize=9)
    for k, (f, name) in enumerate([(f1, "conv1\n32x32"), (f2, "conv2\n16x16"), (f3, "conv3\n8x8")]):
        for j in range(n):
            b = fig.add_subplot(gs[k + 1, j]); b.imshow(norm(f[0, :, :, j]), cmap="gray");
b.set_xticks([]); b.set_yticks([])
            if j == 0: b.set_ylabel(name, fontsize=8, rotation=0, labelpad=24, va="center")
    plt.savefig(fname, dpi=200, bbox_inches="tight"); plt.close()

```

```

# correctly classified examples
pred = model.predict(Xte_f, verbose=0).argmax(1)
ok = np.where(pred == yte)[0]
ex_a = ok[np.where(yte[ok] == 3)[0][0]]; ex_b = ok[np.where(yte[ok] == 8)[0][0]]
show_maps(ex_a, "figs/fig3_maps_digit3.png"); show_maps(ex_b, "figs/fig4_maps_digit8.png")

# Fig 5: class-average of pooled conv3 activations (class 'signature' across channels)
f3_all = fmodel.predict(Xte_f, verbose=0, batch_size=256)
gap = [f.mean((1, 2)) if f.ndim == 4 else f for f in f3_all] # global-average-pooled per
layer
sig = np.stack([gap[2][yte == d].mean(0) for d in range(10)])
fig, ax = plt.subplots(figsize=(8, 3.2)); im = ax.imshow(sig, aspect="auto")
ax.set_xlabel("conv3 channel (64)"); ax.set_ylabel("digit class"); ax.set_yticks(range(10))
ax.set_title("Mean conv3 activation per class"); plt.colorbar(im, ax=ax)
plt.tight_layout(); plt.savefig("figs/fig5_class_signature.png", dpi=200); plt.close()

# Table: per-layer statistics + linear-probe accuracy (how separable are classes at each depth?)
names = ["conv1", "conv2", "conv3", "fc"]; rf = [3, 8, 18, 32]
probe = {}
for k, n in enumerate(names):
    g = gap[k]; ntr = len(g) // 2
    clf = LogisticRegression(max_iter=2000).fit(g[:ntr], yte[:ntr]); probe[n] =
float(clf.score(g[ntr:], yte[ntr:]))
stats = {}
for k, n in enumerate(names):
    f = f3_all[k]
    stats[n] = dict(shape=list(f.shape[1:]), params=int(model.get_layer(n).count_params()),
receptive_field=rf[k], sparsity=float((f == 0).mean()), probe_acc=probe[n])
fig, ax = plt.subplots(figsize=(5.5, 3.2))
ax.bar(names, [probe[n] * 100 for n in names], color="white", edgecolor="black")
for i, n in enumerate(names): ax.text(i, probe[n] * 50, f"{probe[n]*100:.1f}%", ha="center")
ax.set_ylabel("Linear-probe accuracy (%)"); ax.set_ylim(0, 100); ax.set_title("Class separability of
pooled features by depth")
plt.tight_layout(); plt.savefig("figs/fig6_probe.png", dpi=200); plt.close()

json.dump(dict(data=args.data, test_acc=float(test_acc), test_loss=float(test_loss),
train_time=train_time,
epochs=args.epochs, n_train=len(Xtr), n_test=len(Xte), params=int(model.count_params()),
layers=stats, best_val_acc=float(max(hist.history["val_accuracy"])),
final_train_acc=float(hist.history["accuracy"][-1]),
final_val_acc=float(hist.history["val_accuracy"][-1])),
open("results.json", "w"), indent=1)
print(json.dumps(stats, indent=1))

```